BodyScout: Robots That Discover Their Own Bodies

Oct 31, 2026 · @Suyog Neupane

Pitch and research question

One line: BodyScout is a swarm of agents that discovers any jointed robot's body from scratch, without being told whether it is an arm, a gantry, a SCARA or a telescoping boom, plans with what it learned, and rebuilds its self model when the body is damaged, all without ever seeing the robot's model file.

The frontier problem. Every robot works only because an engineer hands it an exact model of its own body. Damage, a new tool, or a reassembly breaks that model, and a human recalibrates by hand. Self modeling robots (Hod Lipson's lab at Columbia) and generative inverse models (Kitchin 2025, *Digital Discovery*) are open research areas. Kitchin names three unsolved problems: choosing which experiments to run, detecting extrapolation, and handling many inputs that give the same output. Inverse kinematics on a redundant arm is exactly that third problem.

What counts as a result.

1. Reaching error in millimeters on targets the swarm never practiced, against the number of motions spent.
2. Swarm vs single agent vs random exploration, same motion budget, same robots. This is the 25% collective capability score.
3. Time and motions to recover after damage (longer link, locked joint, reversed motor, added tool).
4. Transfer: does a skill learned on robot 1 speed up robot 5?

Track: Open Scientific Challenge (A·04), with Physical Agency framing if you add one real servo on Sunday.

The honesty rule that makes it science: agents only send joint commands and receive noisy fingertip observations. They never read the MuJoCo XML. The simulator is the hidden ground truth, so every claim can be scored exactly.

Accounts to set up now

Use the email you registered for the hackathon with on every one of these, or the credits will not attach. Do the first three in the next 20 minutes; the rest only matter if you use them.

- [ ] **Claude Console** ($100 API credits each). Use the event link in the compute PDF, pick "Individual", press "Skip for now" at billing; credits land in 2 to 3 minutes. Create an API key, export it as `ANTHROPIC_API_KEY`. Powers the hypothesizer agent and Claude Code.
- [ ] **GitHub repo.** Create `erSuyognp/bodyscout` (private until judging). Google accounts auto delete Sunday night, so push often.
- [ ] **Google Cloud + AI Studio** (your `@gcplab.me` card; max 2 per team). Sign in from a **dedicated Chrome profile**, not incognito, or you burn the two one time 2FA codes. Copy the Workshop key from aistudio.google.com/apikey into `.env` as `GEMINI_API_KEY`. It gives Gemini 3.6 Flash at 1,000 requests a minute, plus a 32 vCPU VM (c2d-standard-32), Cloud Shell, and Cloud Run for the dashboard. Rules that get projects suspended: never open public ports on a VM (use Cloud Run or `gcloud compute ssh ... -- -L 8080:localhost:8080`), never commit an `AIza...` key, and keep LLM loops bounded (spend throttle about $10 per 10 minutes). Everything is deleted Sunday night.
- [ ] **NVIDIA Brev is the main compute** (the GB10s are all taken). BodyScout runs on CPU, so launch a **Brev CPU instance with 32 to 64 vCPUs** for the sweep: the whole prompt 6 sweep is about 5 core hours, so roughly 10 minutes on 32 vCPUs. A GPU instance is optional, only if you do prompt 13 (Nemotron as a second hypothesizer). Google Cloud's 32 vCPUs become the backup machine.
- [ ] **Brev account** ($150 each, pool the team's). Create an account at brev.nvidia.com, request the coupon via tinyurl.com/brev-credits, redeem with "Add Credits". Stop the instance whenever you are not using it; credits burn by the hour.
- [ ] **OpenAI Platform** ($50 each, via tinyurl.com/openai-credits). Optional: run one modeler agent on GPT to show the swarm is model agnostic.
- [ ] **Nebius Scientific AI** (one request per team, via tinyurl.com/nebius-scientific-AI). Of its three robotics apps, only **Isaac for Healthcare (batch)** fits BodyScout: its `simulate` and `replay` lanes may let you run a medical robot arm in Isaac Lab as a second, unseen body (prompt 12). Skip **Cosmos3-Nano LeRobot augmentation**: it augments imitation learning demo videos, and BodyScout has no demonstrations to augment. Use the interactive Isaac app only to look at the arm before writing the bridge.

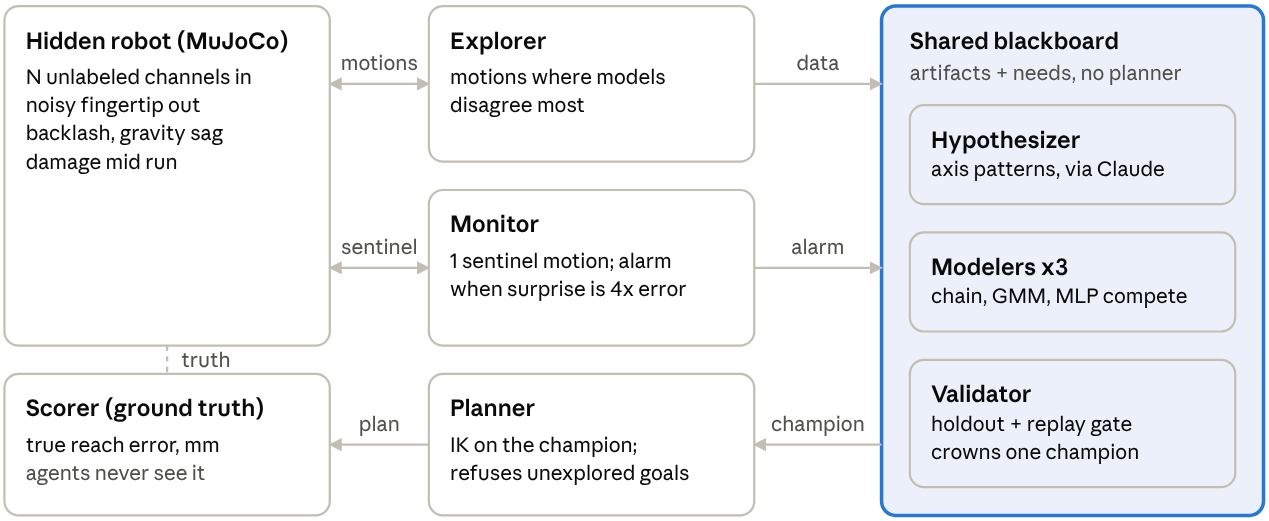
- [ ] **alphaXiv** ($500 GPU credits are a prize, not a sign up). Use the free site for the related work slide.
- [ ] **Robotarium** (robotarium.org). Optional stretch only; approval and queue times are unknown.

Not needed: Formlabs, Micro Center, the hardware pool.

Architecture

Six kinds of agents, plus one channel agent per joint channel when the body type is unknown, share one blackboard and never call each other; the only path to the robot is a motion, and every motion costs budget.

Agents only touch the robot through motions; the scorer alone sees the truth



BodyScout architecture · 6 agents, 1 blackboard, 1 hidden robot

The three gates that make results trustworthy (borrowed from the ScienceClaw self evolution paper, arXiv 2610.08691):

1. **Holdout gate.** One motion in four is held out. Modelers never train on it; the validator scores every candidate on it.
2. **Replay gate.** A candidate is crowned only if refitting it from scratch on the same data reproduces its holdout score (within 1.5x or 2 mm).
3. **Surprise gate.** The monitor compares each sentinel motion with the champion's prediction. Two surprises above 4x the champion's error (floor 8 mm) open a new body epoch: old data stops counting, old parameters become a warm start.

Competing hypotheses, Co Scientist style. The chain modeler is a physics prior (serial joints with unknown axes and offsets). The GMM is Kitchin's generative model of p(q, x), which also samples many joint answers for one target. The MLP is the "just learn it" baseline. They compete on the holdout set; reality picks the winner.

Where Claude sits. The hypothesizer sends Claude 24 raw motions and asks which axis each unlabeled channel turns about. Its answers become starting points for the chain fit. With no API key it falls back to a prior and the skill library, so the system never stalls.

Repo and how to run it

The starter repo (`bodyscout.zip`, sent in chat) already runs end to end on a laptop CPU: 6 unit tests pass, and one swarm run takes 30 to 100 seconds.

Path	What it does
`bodyscout/world.py`	Hidden world: random 3 to 5 joint arms in MuJoCo, 2 mm camera noise, backlash, gravity sag, 6 damage types
`bodyscout/models.py`	Competing self models: chain (physics prior), GMM (Kitchin), MLP
`bodyscout/blackboard.py`	Artifacts with parent lineage, needs with pressure, JSONL provenance export
`bodyscout/agents.py`	Explorer, Hypothesizer (Claude), Modeler x3, Validator, Monitor
`bodyscout/planning.py`	IK on the champion, extrapolation guard, true reach scorer
`bodyscout/swarm.py`	`run(robot, "swarm" / "single" / "mlp")`, learning curves
`experiments/`	learning curve, damage, extrapolation, plots
`demo.py`	Narrated live run, optional MP4 for the backup video
`CLAUDE.md`	Rules for Claude Code, including the never read the robot rule

```
unzip bodyscout.zip && cd bodyscout
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
export ANTHROPIC_API_KEY=sk-ant-...      # optional; hypothesizer falls back
without it
export PYTHONPATH=.
python -m pytest -q tests                # 6 passed
python demo.py                           # arm: learn, damage, recover
python demo.py --family scara            # unknown body: channel agents find
the structure
python demo.py --video results/demo.mp4  # backup video for the slides
python experiments/bodies.py --seeds 3   # headline: unknown body types,
swarm vs single
python experiments/learning_curve.py --arms 6
python experiments/damage.py --arms 3
python experiments/extrapolation.py
python experiments/make_plots.py         # results/fig_*.png +
results/summary.md
```

Push it to GitHub first thing: `git init && git add . && git commit -m "BodyScout starter" && gh repo create bodyscout --private --push --source .`

Timeline

The core already works, so the next 25 hours go to scale, polish, and proof, with the full experiment frozen by Sunday 9 AM.

1. **Sat 10 PM to midnight:** Prompt 4 (ScienceClaw) if not done, then Prompt 15 (identifiability plus rerun) and Prompt 16 (probing baseline). Hand Prompt 18 (Arena) to a cloud Claude Code session in parallel.
2. **Sun midnight to 3 AM:** Prompts 3 and 13 (live Claude and Gemini), then Prompt 17 (knockout test). Teammate continues D2 and D3.
3. **Sun 3 to 7 AM:** Prompt 6 sweep on the Brev CPU instance with every condition (swarm, single, single_probe, knockout). Sleep while it runs.
4. **Sun 7 to 9 AM:** deploy the Arena and post the invite as teams arrive; merge the dynamics branch (D4); freeze results (Prompt 8); record demo.mp4 with --family scara.
5. **Sun 9 AM to 12:30 PM:** slides and talk track (Prompt 9); sit with any team trying the Arena.
6. **Sun 12:30 to 2:30 PM:** rehearse twice, submit slides by 2:30.

Cut rule: must have Prompts 4, 15, 16, 6, 8, 9. Strongly want 17 and 18. Optional: 3 and 13 if keys work, then 19, 20, 21, 22, 5, 11, 12, 14. Humanoids stay on the what is next slide.

Claude Code prompts, in order

Run Claude Code from the repo root so it reads `CLAUDE.md`. Paste each prompt as is; finish one before starting the next, and commit after each.

Prompt 1: orient and verify

```
Read CLAUDE.md and every file in bodyscout/. Then run `PYTHONPATH=. python -m pytest -q tests` and `python demo.py`. Summarize in 10 lines: what each agent does, what the demo printed, and any rule in CLAUDE.md that the code currently violates. Do not change code yet.
```

Prompt 2: GitHub and a CI check

```
Initialize git, add a .gitignore for Python, .env, results/*.mp4 and runs/ (every API key lives in .env, never in code), commit, and create a private GitHub repo named bodyscout with gh, pushing main. Add a GitHub Actions workflow that installs requirements.txt and runs the tests on every push.
```

Prompt 3: live Claude hypothesizer

```
ANTHROPIC_API_KEY is set. In bodyscout/agents.py the Hypothesizer only calls Claude on its second pass in an epoch. Make it call Claude as soon as the channel agents' findings are in (or the epoch has 8 train motions), cache the answer per epoch, and post Claude's one sentence reason as `why` in the hypothesis payload. Give Claude the findings (type, axis, curvature, spread_mm per channel) plus up to 24 raw motions, and ask for structures as tokens like hz or sx. Use the model from env BODYSCOUT_MODEL, a 20 second timeout, and keep the fallback. Show Claude's reason on the hypothesizer line in demo.py. Add an --llm flag to experiments/bodies.py and run 2 seeds per family with and without Claude. Report whether Claude's structures matched robot.spec.tokens() IN THE EXPERIMENT SCRIPT ONLY, never inside an agent.
```

Prompt 4: speak ScienceClaw

```
Clone https://github.com/lamm-mit/scienceclaw into ../scienceclaw, set it up in its own venv with LLM_BACKEND=gemini, GEMINI_MODEL=gemini-3.6-flash and GEMINI_API_KEY from my .env, run `python3 bin/scienceclaw-post --agent BodyScout --topic "robot arm self modeling" --dry-run`, and inspect the Artifact DAG it saves under ~/.scienceclaw/. Then read its README, skills/CONTRIBUTING.md and the artifact / global_index.jsonl format and the needs + ArtifactReactor code. Write bodyscout/scienceclaw_bridge.py that (1) exports our Blackboard artifacts into their artifact schema with parent lineage, appending to ~/.scienceclaw/artifacts/global_index.jsonl, (2) maps our needs to their needs with pressure, and (3) wraps BlackBoxRobot.move as a ScienceClaw skill called `robot_motion` following their skill interface. Keep our own Blackboard as the runtime; the bridge mirrors it. Add a test that round trips 20 artifacts. Explain in 5 lines what a ScienceClaw judge would see.
```

Prompt 5: real robot bodies from MuJoCo Menagerie

```
Add bodyscout/menagerie.py. Clone google-deepmind/mujoco_menagerie (sparse checkout: franka_emika_panda, universal_robots_ur5e, kinova_gen3, trossen_vx300s). For each arm build a BlackBoxRobot subclass that loads the MJCF, uses position control on the arm joints only (ignore grippers), picks the end effector site or the last body as the fingertip, scales joint limits into channels, and exposes the same move / n_channels / limits interface. Hide joint names and the order of channels (shuffle them with a fixed seed). Keep noise, backlash and the damage API working (tool and payload at minimum). Note: these arms have 6 or 7 joints, so ChainModel needs more data; raise budgets to 96 to 160 motions for them. Run them with unknown_body=True so the channel agents find the structure, and report reach error and structure accuracy on two Menagerie arms.
```

Prompt 6: the big sweep

```
Write experiments/sweep.py that runs in parallel across all CPU cores and is resumable (one JSON line per finished job in results/sweep.jsonl, skip finished jobs, print an ETA): (1) bodies: swarm vs single with unknown_body=True on all four FAMILIES, 10 seeds each; (2) damage: swarm vs single for all 6 damage kinds on 5 robots per family, unknown_body=True; (3) learning_curve: swarm, single and mlp on 10 arms with the structure known; (4) extrapolation on 5 robots per family; plus the Menagerie arms if prompt 5 is done. I will run it on a Brev CPU instance with 32 to 64 vCPUs, with a Google Cloud 32 vCPU VM as backup: add the setup and run commands for both to README.
```

Prompt 7: the live dashboard

```
Build dashboard/index.html, a single self contained page (no build step) that loads runs/demo_blackboard.jsonl and results/*.csv and shows: (1) the provenance DAG as an animated timeline of artifacts by agent lane (one lane per channel agent), needs as dashed arrows, the damage moment as a red line; (2) the unknown body types result from results/bodies.csv: reach error and structure found vs truth per body type, swarm vs single; (3) reach error vs motions for swarm vs single vs mlp with medians and min max bands; (4) the champion's discovered structure (tokens like hz sx, link lengths) next to the truth, revealed by a button. Add a Dockerfile serving it with nginx and the gcloud run deploy command for Cloud Run in README. Never open a public port on a VM.
```

Prompt 8: freeze results

```
Run experiments/make_plots.py on the sweep results and produce results/fig_bodies.png, fig_learning.png, fig_damage.png, fig_extrapolation.png, and results/summary.md with every number the slides quote: structure accuracy and median reach error per body type, swarm vs single; median and IQR reach error at 16, 32, 64 motions per condition; detection rate, detection latency and motions to recover per damage kind; refusal rates. Use the exact same robots for every condition. If any baseline beats the swarm somewhere, say where and why in summary.md.
```

Prompt 9: slides content

```
Using results/summary.md and the figures, draft 8 slides as markdown: (1) problem, (2) why it is hard (Kitchin's three open problems, Lipson self modeling), (3) what we built (architecture with channel agents), (4) headline result: any jointed body, structure found 12 of 12 vs 4 of 12, (5) damage recovery, (6) knowing when it is guessing, (7) the honest learning curve on known arms, (8) what is next (hanging humanoids, real servos, the ScienceClaw skill, self driving labs). One claim per slide, each backed by a number from summary.md. Then write a 3 minute talk track that says the honest caveats out loud.
```

Optional prompt 10: one real servo on Sunday

```
Write hardware/esp32_bridge.py: a BlackBoxRobot subclass that sends joint
commands over WiFi to an ESP32 running MicroPython (write that firmware too,
servo PWM with angle limits) and reads the fingertip from a Pi camera
tracking an ArUco marker, converting pixels to metres with a printed grid.
Same move / n_channels / limits interface, so the swarm runs unchanged.
```

Prompt 12: a second simulator through Nebius Isaac for Healthcare

```
First ask the Nebius Scientific AI agent for the exact input and output
schema of the Isaac for Healthcare `simulate` and `replay` lanes, and which
robot arms its scenes contain. Paste that schema into docs/isaac_schema.md.
Then write bodyscout/isaac_replay.py: (1) a script that builds one replay job
of 160 random joint position commands for the chosen arm, plus a second job
with the same commands after attaching an 8 cm tool or changing a link, and
parses the returned end effector positions into results/isaac_<arm>.npz; (2)
a ReplayRobot class with the same move / n_channels / limits interface that
answers from those recorded pairs (the explorer may only pick commands that
were recorded). Run the swarm and the single agent on it and add the result
to summary.md as "same swarm, unseen simulator, unseen medical arm". If the
lanes cannot accept joint commands and return the end effector pose, stop and
tell me instead of faking it.
```

Prompt 13: three model families in the swarm (Claude, Gemini, Nemotron on Brev)

```
On the Brev instance, serve an open Nemotron model with vLLM behind an OpenAI
compatible endpoint (give me the exact commands, I will run them). Then add a
backend option to the Hypothesizer in bodyscout/agents.py:
BODYSCOUT_LLM=claude|gemini|nemotron|all, reading GEMINI_API_KEY (model
gemini-3.6-flash) and NEMOTRON_URL; skip Nemotron cleanly if NEMOTRON_URL is
unset. Cap every LLM agent at 1 call per epoch. With `all`, post each model's
patterns as separate hypothesis artifacts tagged by source, and let the
validator's results show which source proposed the axes that won. Run 3 arms
with each backend and add a small table to summary.md: how often each model's
first guess matched the true axes (checked in the experiment script only,
never inside an agent).
```

Experiments and first numbers

First results, already measured here on 3 to 4 random arms (3 to 5 joints, 2 mm camera noise): the swarm recovers from 3 of 4 injuries to 2 to 3 mm while a single agent stays 25 to 44 mm off, and its planner refuses every out of reach target. Rerun at scale tonight

(prompt 6) before quoting these on slides.

1. Learning an unknown body (4 arms, median reach error on unseen targets, mm)

Motions	Swarm	Single agent, chain model	Single agent, neural net
16	9.6	4.0	363
24	3.3	2.6 (one fit was 215 mm off)	289
32	3.0	2.2	143
64	1.7	1.6	123

On a clean arm the single chain agent is slightly faster, because the swarm spends a quarter of its motions on validation. The 215 mm outlier is a bad local minimum the single agent accepted unchecked; the swarm's replay gate exists to reject exactly that.

2. Damage at motion 48 (3 arms per type, error at motion 112, mm)

Damage	Swarm noticed	Swarm	Single agent
Link 60% longer	3 of 3 runs	3.4	24.8
Motor rewired backwards	3 of 3	3.2	43.5
8 cm tool added	3 of 3	2.2	33.2
Bracket bent 25 degrees	0 of 3	13.1	8.1

The bent bracket is the honest miss: it shifts the tip by less than the monitor's 8 mm alarm floor, so the swarm keeps trusting a stale model. Fix it tonight with prompt 11 below.

3. Knowing when it is guessing (3 arms, 10 reachable and 10 out of reach targets each)

Targets	Swarm planner refused	Naive planner's median miss
Reachable	0%	2 mm
Out of reach	100%	326 mm

Prompt 11: catch subtle damage

```
The Monitor in bodyscout/agents.py misses a 25 degree bent bracket because
each sentinel surprise stays under the 8 mm floor. Replace the two strikes
rule with a CUSUM detector on the sentinel residuals, normalized by the
champion's holdout error, and let the monitor raise its own sentinel rate
(every tick instead of every third) for 6 ticks after a near miss. Rerun
experiments/damage.py --kinds bent_axis long_link and report detection rate
and false alarms on undamaged runs (run learning_curve and count alarms).
Keep false alarms at zero on the 4 undamaged arms.
```

One system for any jointed body

Told nothing about the body type, the swarm found the exact structure of all 12 robots (4 types x 3 each) and reached 1 to 2 mm; a single agent searching blind found 4 of 12 and stayed 35 to 214 mm off.

How it works. One channel agent per unlabeled channel sweeps only its own channel and reads the shape of the fingertip path: a straight line means the joint slides, an arc means it rotates, and the line direction or the arc's plane gives the axis. No channel agent sees the whole body. The hypothesizer assembles their findings into whole body structures (`hz` = rotates about z, `sx` = slides along x), and the modelers and validator take it from there.

Result (median reach error after 64 motions, mm; structure found)

Body	Swarm	Single agent	Swarm structure right	Single structure right
Arm (all rotating)	2.1	214	3 of 3	1 of 3
Gantry (3 slides + wrist)	1.1	42	3 of 3	1 of 3
SCARA (2 rotating, 1 slide, wrist)	1.3	35	3 of 3	1 of 3
Telescoping boom	0.9	48	3 of 3	1 of 3

The slide claim: "The less you know about the body, the more the swarm helps." On arms with the structure handed over, single and swarm were within 0.1 mm. With the type unknown, the gap is 30 to 100 times.

Say these honestly if asked.

- The win comes from splitting the evidence channel by channel. A single agent programmed to probe one channel at a time would close much of the gap; the swarm makes that split natural and parallel, and that is the point to defend.
- The swarm pays for it: 16 to 20 probe motions out of 64, and about 7 times the compute per run (about 100 s vs 15 s on 2 CPU cores).
- Branching bodies (hands, two arm setups), wheeled robots and walking robots are not covered yet.

Run it: `python experiments/bodies.py --seeds 3`, then `python demo.py --family scara` for the live version.

Prompt 14: rerun everything in unknown body mode, then branch

```
Read experiments/bodies.py and the ChannelAgent in bodyscout/agents.py. (1)
Rerun experiments/damage.py and experiments/learning_curve.py with
unknown_body=True across all four FAMILIES (add a --unknown flag), 5 seeds
each, on the Brev CPU instance, and update make_plots.py and summary.md. (2)
Then add branching bodies: a world family `twoarm` with two serial limbs from
a shared base, each with its own fingertip marker, so move() returns a (2, 3)
array. Generalize ChainModel to a tree where each channel belongs to one
limb, and let channel agents decide which marker their channel moves. Keep
the old single marker path working, keep the never read the robot rule, and
report structure accuracy for twoarm the same way.
```

Teammate track: learning dynamics

Goal by Sunday 9 AM: one slide showing that a 350 g payload, invisible to the shape model, is caught and measured by a learned dynamics model, plus better prediction of fast motions. It all lives on its own branch and in new files, so it never blocks the main track.

Ground rules

- Branch `dynamics`. New files only: `bodyscout/dynamics/`, `experiments/dynamics.py`, `tests/test_dynamics.py`. Do not edit `agents.py`, `models.py`, `world.py` or `swarm.py`; one small hook gets merged Sunday at 8 AM.
- The never read the robot rule still applies: agents see only commanded joint targets over time and the tip position over time.
- Contacts are disabled in the sim, so gradients through MuJoCo MJX stay smooth. Use short rollouts (0.3 to 0.5 s) so gradients do not explode.

- Start on CPU (JAX runs there); move to a Brev GPU instance once you batch many rollouts with `vmap`.

By	Milestone	Fallback if stuck
Sat 7:30 PM	Branch, environment, tests pass (D0)	Ask the main track; nothing depends on it
Sat 9:30 PM	Robots that move over time with hidden masses, damping, motor strength (D1)	Fewer knobs: payload and damping only
Sun 12:30 AM	Fit those knobs by gradient descent through MJX (D2)	CPU MuJoCo plus scipy least squares over 3 to 5 parameters
Sun 3:00 AM	Dynamics agent, payload detection, figure (D3)	Skip the agent; keep the experiment and figure
Sun 8:00 AM	Merge the hook, numbers into summary.md	Present it from the branch as its own slide

Prompt D0: set up

```
Read CLAUDE.md, bodyscout/world.py, bodyscout/models.py and
bodyscout/agents.py so you understand the black box rule, the ArmSpec,
ChainModel and the blackboard. Create and switch to a git branch named
dynamics. Add jax and mujoco-mjx to a separate requirements-dynamics.txt (CPU
jax by default; note the jax[cuda12] line for Brev GPU). Create the empty
package bodyscout/dynamics/ with __init__.py and tests/test_dynamics.py with
one smoke test that loads an MJX model from build_mjcf(random_body(1, "arm"))
and steps it. Run all tests. Do not edit any existing file except adding the
new requirements file. Commit.
```

Prompt D1: robots that move over time

```
In bodyscout/dynamics/world_dyn.py write DynamicRobot, a subclass of
BlackBoxRobot that adds hidden dynamics: per link mass multipliers (0.6 to
1.6), payload mass at the tip (0 to 0.4 kg), per joint damping (0.1 to 1.0)
and per joint servo stiffness kp (30 to 90), all drawn from a seed and stored
only in the hidden spec. Agents may use only these: n_channels, limits, dt,
and move_trajectory(targets) where targets is a (T, n) array of joint
commands at dt = 0.01 s; it returns the (T, 3) tip path with 2 mm noise,
starting from rest at targets[0], and counts T/50 motions of budget per call.
Do not change move(). Add damage kinds payload_dyn (+0.35 kg) and weak_motor
(one joint kp x 0.5). Add tests: same seed gives identical paths; a payload
visibly changes the tip path during fast moves; the agent facing methods
never expose the hidden spec. Do not edit world.py. Commit.
```

Prompt D2: fit dynamics by gradient descent through MJX

```
In bodyscout/dynamics/sysid.py write fit_dynamics(mjcf_xml, trajectories,
observed_paths) that estimates payload mass, per link mass multipliers, per
joint damping and per joint kp by minimising tip path error. Build the model
with MJX, roll it out with jax.lax.scan, vmap over trajectories and over 16
random initial guesses, optimise log parameters with optax Adam for a few
hundred steps, and keep the best guess. Use rollouts of 0.3 to 0.5 s chopped
from longer trajectories so gradients stay stable, and clip gradients. First
test with the TRUE kinematic MJCF (allowed in experiments only); then add
chain_to_mjcf(model) that builds the MJCF from a fitted ChainModel's
structure tokens and parameters, and fit on that. If MJX gradients are
unstable or too slow, fall back to CPU MuJoCo rollouts with scipy
least_squares over payload, damping and kp only. Report parameter error vs
truth and tip path error on unseen fast trajectories, comparing: shape only
(quasi static ChainModel prediction), learned dynamics, and true dynamics.
Commit.
```

Prompt D3: the agent, payload detection and the figure

```
Add bodyscout/dynamics/agents_dyn.py with two blackboard agents that follow
the same rules as the others (artifacts with parents, no robot attributes
beyond the allowed ones): DynamicsModeler requests fast excitation
trajectories through a need, fits dynamics with fit_dynamics on top of the
current kinematic champion, and posts a dynamics_model artifact with its
holdout path error; DynamicsMonitor runs one short fast sentinel trajectory
every few ticks and declares a change when the path error jumps, naming the
most likely cause by refitting only payload, then only each joint's kp, and
reporting which refit explains the jump best. Write experiments/dynamics.py:
on 3 arms and 2 SCARAs, add payload_dyn or weak_motor at motion 48 and
compare the existing shape Monitor vs the DynamicsMonitor on detection rate,
motions to detect, and whether the named cause and estimated payload mass are
right. Produce results/fig_dynamics.png (tip path error on fast moves: shape
only vs learned dynamics vs truth) and a summary_dynamics.md table. Commit.
```

Prompt D4 (Sunday 8 AM): merge

```
Rebase the dynamics branch on main. Add the smallest possible hook:
make_swarm(..., dynamics=False) in bodyscout/swarm.py that, when True, adds
DynamicsModeler and DynamicsMonitor; nothing else in shared files changes.
Run the full test suite and the main demo twice to confirm main results are
unchanged and deterministic. Append summary_dynamics.md to
results/summary.md. Open a pull request.
```

The slide this produces: "Shape alone cannot see a 350 g payload. The dynamics agent catches it, names it and weighs it." State the estimated vs true mass, and say that it is still simulation.

Prompts added Saturday evening

These came out of tonight's review. Two are already done; run the rest in the order of the timeline above.

- [x] ~~**Rule fixes:** Monitor no longer reads robot.motions, hypotheses cite parents, champion scores are new artifacts (done).~~
- [x] ~~**Determinism:** ChainModel.fit always uses trf; two runs give identical output (done).~~
- [ ] Prompt 15: plausible link lengths and identifiability, then rerun everything
- [ ] Prompt 16: single agent with probing baseline
- [ ] Prompt 17: no single points of failure, plus the knockout test
- [ ] Prompt 18: BodyScout Arena for other teams (+10 bonus)

- [ ] Prompt 19: bodyscout learn, URDF and MJCF export
- [ ] Prompt 20: does it learn faster as more robots use it
- [ ] Prompt 21: markerless vision on a Brev GPU
- [ ] Prompt 22: brute force GPU baseline (optional)

Prompt 15: plausible link lengths and identifiability, then rerun

```
Two changes to ChainModel in bodyscout/models.py. (1) Append a small penalty
to the least_squares residuals that pulls each offset vector and the tip
vector toward a short plausible length (weight 1e-3 on norms above 0.3 m,
zero below). Confirm with experiments/bodies.py (2 seeds per family) that
reach error and structure accuracy do not get worse and no reported link
exceeds 0.5 m. (2) After the fit, compute the residual Jacobian at the
solution and mark a link length not identifiable when moving it along a near
null direction (singular value below 1e-6 of the largest) changes predicted
tips by less than 0.5 mm over the training motions. Add describe()
['identifiable'] per link, print 'not identifiable' in demo.py for those, and
test that SCARA vertical offsets are not identifiable while arm links are.
Diff two runs of each demo to confirm determinism. Then rerun bodies, damage,
learning_curve and extrapolation, regenerate figures with make_plots.py, and
list every number that changed versus the previous summary.md. Commit.
```

Prompt 16: single agent with probing baseline

```
Add a condition single_probe to bodyscout/swarm.py: one sequential agent that
first sweeps each channel one at a time with the Explorer's probe values,
classifies each channel with the same rule ChannelAgent uses, builds the
structure itself, then fits ChainModel on all its data with no holdout,
validator or monitor. Same motion budget and scorer. Add it to
experiments/bodies.py and experiments/damage.py (unknown_body=True), rerun 3
seeds per family, and report structure accuracy, reach error, motions until
the first good model and wall clock time for swarm vs single_probe vs single.
State plainly in summary.md where single_probe ties or beats the swarm.
Commit.
```

Prompt 17: no single points of failure, plus the knockout test (run after prompts 3 and 13)

```
Remove single points of failure. (1) Split the Hypothesizer into independent agents that each post hypothesis artifacts tagged by source: FindingsHypothesizer, PriorHypothesizer, LibraryHypothesizer, and LLM hypothesizers for Claude and Gemini when keys exist; none waits on another. (2) Let each chain Modeler assemble structures directly from channel findings when no hypotheses exist. (3) Make crowning a public rule: champion = lowest holdout error among candidates that pass the replay check, computed by a pure function crown(bb) any agent can call; run two Validators and post the champion only when both compute the same id, logging disagreements. (4) Allow two Explorers sharing the robot through needs. (5) Add experiments/knockout.py: 3 seeds x 4 families in unknown body mode, remove one agent type at motion 24 (each hypothesizer type, one validator, one explorer, the monitor) and report structure accuracy and reach error vs the full collective. Keep all rules: immutable artifacts with parents, agents touch only move, n_channels, limits, deterministic runs. Commit.
```

Prompt 18: BodyScout Arena for other teams

```
Build the BodyScout Arena so other teams can test their agents on our hidden robots. (1) arena/server.py, a FastAPI app: POST /session {family, seed, damage?} returns a session id plus n_channels and limits only; POST /session/{id}/move {q} returns the noisy tip and motions used; POST /session/{id}/submit scores reach error on hidden targets plus structure tokens if given; GET /leaderboard. Hidden specs never leave the server; cap 128 motions per session; rate limit per team name; no keys in code. (2) arena/client.py: a 20 line client exposing move, n_channels and limits so any BlackBoxRobot agent runs unchanged, plus a JSON tool schema for LLM agents. (3) examples/random_agent.py and examples/bodyscout_agent.py. (4) Dockerfile and the gcloud run deploy command; deploy from Google Cloud Shell, never by opening a VM port. (5) A one page leaderboard at GET /. Log every team's session as evidence of reuse. Tests run a full session locally. Commit.
```

Prompt 19: bodyscout learn, URDF and MJCF export

```
Package BodyScout with pyproject.toml so pip install from GitHub works, and add a CLI: 'bodyscout learn --robot path.py:Class --budget N' runs the swarm in unknown body mode and writes robot.urdf and robot.xml (MJCF) from the champion ChainModel (one link and joint per channel, revolute or prismatic from the structure tokens, axes and origins from the fit, a tip frame, placeholder masses with a comment that the file is kinematics only, non identifiable lengths marked in a comment), plus body.json and report.html; 'bodyscout watch' runs only the Monitor and alerts on body changes; 'bodyscout bench --agent path.py:Class' scores someone else's agent on the hidden robots. Add bodyscout/adapters/ with a documented RobotAdapter base class, a MuJoCo adapter, and an ArUco camera template; add load_body(). Test that robot.xml loaded back into MuJoCo matches ChainModel.predict within 1 mm on 50 random commands. Add a 'Use it on your robot' README section. Commit.
```

Prompt 20: does it learn faster as more robots use it

```
Add experiments/library.py. For each family, make 8 robots with different seeds (same structure, different lengths, tilts and offsets, like copies of one product). Run them in sequence cold (each from scratch) and shared (after each robot, add its champion structure to the hypothesizer library and pass its fitted parameters as a warm start to the next robot's chain modeler). Record motions until median reach error first drops below 3 mm, and final reach error, for robots 1 to 8, and plot both curves. Add one wrong prior case (library from another family) and confirm accuracy does not get worse, only slower. Agents never read robot.spec. Commit and add the numbers to summary.md.
```

Prompt 21: markerless vision on a Brev GPU

```
On a Brev GPU instance render MuJoCo images with MUJOCO_GL=egl from two fixed cameras at 640x480. Write bodyscout/vision.py with a CameraRobot wrapper (same move, n_channels, limits) that returns a 3D tip estimated from images only: default detector Gemini Robotics ER via GEMINI_API_KEY asked to point to the end of the last link in each image; fallback detector color segmentation of the red tip sphere. Triangulate with the known camera matrices (camera calibration allowed, robot geometry not). Cache images and detections in runs/vision/. Report detection error vs the true tip, rerun experiments/bodies.py with CameraRobot on 2 seeds per family, and compare with the marker version. Save a short video of both camera views with detected points drawn. Commit.
```

Prompt 22: brute force GPU baseline (optional)

```
Add experiments/bruteforce_gpu.py using JAX. Reimplement ChainModel forward
kinematics in jax.numpy with hinge and slide joints, fit all structures (2
types x 3 axes per channel) in parallel with vmap and a batched Gauss Newton
or Adam loop from several inits, on the same random motions the single agent
collects, and pick the structure with the lowest holdout error. Compare with
swarm and single on the experiments/bodies.py robots: structure accuracy,
reach error, motions used, GPU seconds. Fall back to CPU so tests pass.
Report honestly if brute force ties or wins. Commit.
```

Teammate track: markerless vision (Prompt 21, split into steps)

Goal by Sunday 8 AM: the swarm learns a body from two camera images per motion instead of a perfect marker, with an honest accuracy number and a short video. Branch `vision`, new files only (`bodyscout/vision.py`, `experiments/vision.py`, `tests/test_vision.py`); shared files are touched only by the merge step.

This track runs on a Brev GPU instance only. Running it on a CPU instance, or rendering on the CPU, is not acceptable. If the GPU does not work, fix the GPU setup; do not fall back.

Brev setup, step by step

1. **Install the Brev CLI on your laptop and log in:** `brew install brevdev/homebrew-brev/brev && brev login` (on Linux use the install script from the Brev docs).
2. **Create a GPU instance** in the Brev web console with your own $150 credits. Pick an NVIDIA GPU instance (an L4 or similar is plenty for rendering); never a CPU only instance. Then run `brev refresh` and `brev ls` to see it by name.
3. **Connect:** `brev open <instance-name> cursor` (or `code`) opens an editor running on the GPU machine; for a plain terminal use `brev shell <instance-name>`.
4. **Confirm the GPU before anything else:** run `nvidia-smi` on the instance. It must list an NVIDIA GPU. If it does not, you are on the wrong instance type: stop it and create a GPU one.
5. **Get the code:** `git clone https://github.com/erSuyognp/bodyscout && cd bodyscout && git checkout -b vision` (the repo is public, so cloning needs no login).
6. **Copy the keys from your laptop** (they are never in git): `scp .env <instance-name>:~/bodyscout/.env`. It must contain `GEMINI_API_KEY`.
7. **Install Claude Code on the instance:** `npm install -g @anthropic-ai/claude-code`, then run `claude` in the repo folder and log in with the link it prints.

8. **Work inside tmux** so a closed laptop does not kill a run: `brev open <instance-name> tmux`, or type `tmux` in the shell.
9. **Push permission:** Suyog adds you as a collaborator on GitHub (repo Settings, Collaborators); then run `gh auth login` on the instance and push only to the `vision` branch.
10. **Big files back to your laptop:** `scp <instance-name>:~/bodyscout/results/vision_demo.mp4 .`
11. **When done, stop the instance in the console** (credits burn by the hour). After a restart run `brev refresh`, because its address may change.

By	Step	If stuck
Sun 12:30 AM	V0: GPU confirmed, rendering on the NVIDIA GPU via EGL	Fix drivers or EGL libraries, or switch to a different GPU instance image; never render on CPU
Sun 1:30 AM	V1: two cameras, color marker detector, triangulation, accuracy	None needed; this is the baseline
Sun 3:00 AM	V2: no marker, Gemini Robotics ER finds the hand	Report its accuracy honestly even if worse
Sun 4:30 AM	V3: swarm learns from cameras, video	Run 1 seed per family only
Sun 8:00 AM	V4: merge with the dynamics branch	Present from the branch

Prompt V0: set up and prove rendering

```
Read CLAUDE.md and bodyscout/world.py. Create branch vision from main. This work must run on the NVIDIA GPU; CPU rendering (osmesa) is not allowed as a fallback. First run nvidia-smi and stop with a clear message if no NVIDIA GPU is present. Set MUJOCO_GL=egl, render one 640x480 image of random_body(1, 'arm') with mujoco.Renderer, save it to runs/vision/check.png, and prove the GPU did the work: print the OpenGL vendor and renderer strings from the active EGL context (they must name NVIDIA) and show the python process in nvidia-smi during a 200 frame render loop; report frames per second. If EGL fails, diagnose and fix it on the instance (NVIDIA driver, libEGL and the NVIDIA EGL vendor file, PYOPENGL_PLATFORM=egl) instead of switching backends. Make bodyscout/vision.py raise an error at import time if MUJOCO_GL is not egl. Add tests/test_vision.py with a render smoke test that also asserts the renderer string names NVIDIA (skip only when no GPU is present). Do not edit existing files. Commit.
```

Prompt V1: two calibrated cameras, the marker baseline

```
Write bodyscout/vision.py. Add two fixed cameras (front and side, both seeing the whole workspace) by wrapping the scene, not by editing world.py. Write a CameraRobot class with the same move, n_channels and limits interface as BlackBoxRobot: it moves the hidden robot, renders both views, detects the tip in each image, and triangulates a 3D point using the camera matrices computed from the MuJoCo camera parameters (camera calibration is allowed; robot geometry is not). Detector 1: color segmentation of the red tip sphere, sub-pixel centroid. Cache every image and detection under runs/vision/ keyed by a hash of the image. Measure detection error vs the true tip on 200 random poses per family, plus how often the tip is hidden or out of view, and report median and 90th percentile in mm. Commit.
```

Prompt V2: no marker, Gemini finds the hand

```
Add a markerless mode to CameraRobot: render the tip sphere the same color as the links so there is no marker. Detector 2: ask Gemini Robotics ER (find the exact model id by listing models with GEMINI_API_KEY; the hackathon key includes the Robotics ER preview) to point to the end of the robot's last link in each image, returning normalized 2D points; triangulate as before. Cache responses by image hash so reruns are free and repeatable, cap total calls per run, retry once on errors, and fall back to detector 1 only in marker mode. Report detection error vs truth for markerless Gemini vs the color marker on the same 200 poses per family, plus how often Gemini's two views disagree badly (triangulation residual above 20 mm). Commit.
```

Prompt V3: the swarm learns from cameras, and the demo video

```
Write experiments/vision.py: run the swarm in unknown body mode on
CameraRobot (markerless Gemini and color marker) for 2 seeds per family,
budget 64, and compare with the normal marker robot on structure found and
reach error (scored on the true tip, as always). Also let the Monitor treat a
large triangulation residual as a bad observation rather than a body change.
Produce results/fig_vision.png and summary_vision.md, stating plainly where
camera input makes things worse. Render a 20 second video of one scara run:
both camera views side by side with the detected point drawn, and the agents'
narration lines as captions; save to results/vision_demo.mp4. Commit.
```

Prompt V4 (Sunday 8 AM): merge

```
Rebase the vision branch on main after the dynamics branch is merged. Add
only a flag to demo.py (--camera marker|gemini) that swaps in CameraRobot;
nothing else in shared files changes. Run the full test suite and diff two
demo runs (cached detections make them repeatable). Append summary_vision.md
to results/summary.md. Open a pull request.
```

The slide this produces: "No marker, just two cameras: the swarm still finds the structure, with reach error X mm vs Y mm with a marker." Use the real numbers, whichever way they go.

Demo script and slides

Live demo, 90 seconds. Run `python demo.py --arm 3 --damage long_link` with the terminal font large, and the dashboard open beside it.

1. "This robot has 5 unlabeled channels. The swarm gets nothing else: no model file, no joint names."
2. Point at the hypothesizer line and the first crowned champion: "After about 24 motions it has discovered its axes and link lengths; here they are next to the truth."
3. "Now I secretly swap a link for one 60% longer." Point at the monitor alarm: "It noticed on its own, declared its body changed, and stopped trusting its old self."
4. Point at the paused gap and the recovery: "It refuses to move until it has a new model it can trust, then it is back to about 3 mm. The single agent baseline is still 34 mm off."
5. Close on the provenance file: "Every number traces back to the raw motions that produced it."

Then run `python demo.py --family scara` for 30 seconds: "Now it is not even told what kind of robot this is. Each channel agent wiggles one joint; a straight line means it slides, an arc means it rotates." Point at the four findings and the crowned structure. If the live run misbehaves, play `demo.mp4` and say so plainly.

Seven slides.

#	Slide	The one claim
1	Robots do not know their own bodies	Every robot ships with a hand written model; damage and tool swaps break it
2	Why it is hard	Kitchin's three open problems plus Lipson's self modeling, on real physics
3	BodyScout	Channel agents, competing modelers, three gates, one blackboard, no planner
4	Headline: any jointed body	Told nothing about the body type: structure found 12 of 12 vs 4 of 12 for a single agent
5	Damage	The swarm notices, pauses and relearns; the single agent never recovers
6	Knowing when it is guessing	Refuses every out of reach target; the naive planner misses by 326 mm
7	The honest baseline	With the arm structure handed over, a single agent is as good: the swarm helps most when you know least
8	What is next	Hanging humanoids, real servos, the first ScienceClaw skill that moves a body, self driving labs

Say the honest part out loud. On a clean, undamaged arm, one agent that already knows to use a chain model learns slightly faster, because the swarm spends one motion in four on validation. That validation is exactly what lets the swarm notice damage, which the single agent never does. Judges from the organizing lab will trust you more for saying it first.

Risks and fallbacks

Risk	What you will see	Fallback
Judges say "it is only simulation"	Questions about physical agency	Agents only get a noisy marker, backlash and gravity sag; the ESP32 bridge (prompt 10) runs the same swarm on a real servo
Judges say "a single agent with the right model is enough"	Result 1 is close on clean arms	Agree for arms whose structure is handed over, then show the unknown body result (12 of 12 vs 4 of 12) and damage recovery
Menagerie arms need too many motions	Chain fits on 6 to 7 joints stay above 10 mm at 64 motions	Raise budget to 160, or keep them as a stretch slide
Claude call fails or is slow	Hypothesizer logs `llm unavailable`	Library plus prior still work; the swarm never stalls
Sweep runs out of time	Fewer arms than planned	4 arms and 4 damage kinds already give the headline numbers
Live demo crashes	Stack trace on stage	Play `demo.mp4`, show the dashboard recording
Someone asks about rendering	No 3D MuJoCo viewer in slides	`python -m mujoco.viewer` on your laptop for a 10 second clip; the cloud box here has no GPU renderer

Known limits to state if asked: a body change is detected after 6 to 15 motions because the monitor checks every third tick; payload changes cause only millimetres of sag and are hard to detect; the GMM and MLP rarely win at these budgets, which is itself a finding about physics priors.